Team plan: graph models with and without the V columns

Oct 21, 2026 · @Reset

Why this study

Across 3 random seeds, THGNN's graph and payment history add nothing over the same network without them, with the V columns and without them. Our second model shows the same: adding GraphSAGE to the FT-Transformer does not raise its score. What is left to find out is whether this holds for other graph models too, or only for the two we built.

Model (test set)	ROC-AUC	PR-AUC
THGNN, with V (3 seeds)	0.843 (0.839–0.847)	0.392 (0.380–0.401)
Same network without graph or history, with V (3 seeds)	0.841 (0.834–0.848)	0.410 (0.394–0.428)
THGNN, without V (3 seeds)	0.773 (0.772–0.774)	0.187 (0.173–0.201)
Same network without graph or history, without V (3 seeds)	0.776 (0.767–0.781)	0.215 (0.207–0.221)
FT-Transformer alone (1 seed)	0.819	0.383
FT-Transformer + GraphSAGE (1 seed)	0.818	0.380
GraphSAGE alone (1 seed)	0.741	0.138
TabNet, the SOA member's rebuild of the paper's model (all 431 columns, 1 seed)	0.852	0.395
Reference paper: TabNet	0.759	0.149
Reference paper: TabNet–GraphSAGE hybrid	0.791	0.206
Reference paper: LightGBM	0.834	0.394

Seed results are the mean, with the lowest and highest seed in brackets. Reference paper: Ahmad et al. (2026), *Scalable and interpretable fraud detection in FinTech*, J. Finance & Data Science 12, 100203. On validation, the FT-Transformer with a randomly shuffled graph

scored the same as with the real graph (PR-AUC 0.489 vs 0.490). A TabNet built following the paper's setup scores far above the TabNet the paper reports.

Research questions

The study answers three questions, each with four graph models (GraphSAGE, GAT, R-GCN, HAN) instead of one.

1. **RQ1. Does a graph model add anything?** Does adding a graph model raise the score above the tabular model alone?
2. **RQ2. Are the V columns the reason?** If the V columns already hold what the graph knows, the graph should start to help once they are removed (column set B).
3. **RQ3. Is the graph information real?** A real graph must beat the same graph with its links shuffled at random.

Fixed setup for every member

Everyone uses exactly the same data, split, graph and training rules, so any difference in scores comes from the model and nothing else. Nobody changes these without the whole team agreeing.

Item	Rule	Source
Data	IEEE-CIS `train_transaction` + `train_identity`, joined on TransactionID: 590,540 payments	Paper, Section 3.1
Split	Sorted by TransactionDT, 70 / 15 / 15: train 413,378 (days 1–121), validation 88,581 (days 121–152), test 88,581 (days 152–183). Everyone loads Reset's `ieee_sorted_split.parquet`; nobody makes their own split	Paper, Section 3.2
Column set A	The reference paper's Table 2: TransactionAmt, card1–6, addr1–2, P/R_emaildomain, DeviceType, DeviceInfo, id_01–id_38, V1–V339 (390 columns)	Paper, Table 2
Column set B	Set A without V1–V339 (51 columns)	This study's test

Item	Rule	Source
Preprocessing	Missing values filled with training medians, scaled with training mean and std, categories learned from training only, unseen values become "unknown"	Paper, Section 3.2
Graph	One shared entity graph, built by the SOA member from the split file (see "Mandatory settings for all graph models")	Paper, Section 3.3.2
Optimizer	Adam	Paper, Table 3
Loss	Weighted binary cross-entropy, fraud weight √(N0/N1) from the training set	Paper, Table 3
Model selection	Early stopping on validation ROC-AUC, patience 10. The test set is scored once, at the end of each run	Paper, Table 3 (patience: team rule)
Tabular epochs	Up to 80	Paper, Table 3
Seeds	42, 1 and 2 for every run (the paper uses one fixed seed; seed 42 is ours)	Team rule
Saved score	The raw model output before the sigmoid (logit)	Team rule

Mandatory settings for all graph models

GAT, R-GCN, HAN and the paper's hybrid must all use exactly these settings, so the only difference between them is the graph layer itself. GraphSAGE uses the same SOA graph file, and the FT-Transformer teammate is free to test other settings for it. Anything the paper does not give is chosen once by each member, kept the same for all seeds, and written in the run's history file. THGNN keeps its own payment graph inside the model and is not affected.

Setting	Mandatory value	Source
Graph file	The SOA member's entity graph, built from Reset's split file: 14,239 nodes (including one "missing" and one "unknown" node per entity type) and 12,401,340 links before repeats are removed. Nobody builds their own graph	Team rule (paper: 14,298 nodes, about 12.4 million links)
Entity columns	card1 (account), addr1 (address), P_emaildomain and R_emaildomain (email), DeviceInfo and DeviceType (device)	Paper, Section 3.3.2 (account, address, email, device)
Links	Entities that appear in the same training payment are linked to each other; message passing uses each link once	Paper, Section 3.3.2
Built from	Training payments only	Paper, Section 3.2
Missing values	Their own "missing" node per entity type	Team rule (this gives the paper's 12.4 million links)
Unseen values	Mapped to the "unknown" node of their entity type	Paper, Sections 3.2 and 3.3.2
Unknown-node training	In each epoch, 10% of the training payments' entities are replaced by their "unknown" node, so that node learns something	Team rule (as in the SOA GraphSAGE)
Node inputs	A learned embedding for each node plus an embedding for its entity type	Team rule (as in the SOA GraphSAGE)
Payment columns	None: no V columns and no other payment columns	Team rule (each graph model is then fused with both column sets)
Payment score	The payment's 6 entity embeddings are joined (6 × 64 = 384) and passed through Linear(384, 128) → ReLU → dropout → Linear(128, 1)	Team rule (as in the SOA GraphSAGE)

Setting	Mandatory value	Source
Number of layers	2	Team rule (the paper does not say)
Embedding size	64	Paper, Table 3
Epochs	12, keeping the epoch with the best validation ROC-AUC	Paper, Table 3
Optimizer	Adam	Paper, Table 3
Loss	Weighted binary cross-entropy, fraud weight $\sqrt{}$(N0/N1) from training payments	Paper, Table 3
Seeds	42, 1 and 2	Team rule
Shuffled control	The same model on the SOA member's shuffled link file for the same seed	Team rule
Test set	Scored once, at the end of each run	Team rule
Saved scores	Logit for every validation and test payment, with TransactionID and isFraud	Team rule
Free choices	Learning rate, dropout, attention heads (GAT, HAN) and weight sharing (R-GCN): chosen once, the same for all seeds, written in the history file	Not in the paper

The team

Five members, each with their own models, all working at the same time. New members 1 and 2 and the SOA member each build one of the three graph models that are left, and the SOA member also brings TabNet, the reference paper's own tabular model.

Member	Role	Models	Training runs
Reset	THGNN; tabular baselines; sends the split file; fusion and evaluation; puts the paper together	THGNN (done), plain network, LightGBM, graph-features baseline	14 left (THGNN's 12 are done)
FT-Transformer teammate	Tabular base and GraphSAGE on the SOA graph	FT-Transformer, GraphSAGE	12
New member 1	Graph model	GAT	6
New member 2	Graph model	R-GCN	6
SOA member	Builds and shares the graph file; graph model; the paper's tabular model and the paper's own hybrid	HAN, TabNet, TabNet–GraphSAGE hybrid	18

- **GraphSAGE:** each payment's score is built from the average of its neighbours (the reference paper's graph encoder).
- **GAT:** like GraphSAGE, but the model learns how much weight each neighbour gets (attention).
- **R-GCN:** a separate set of weights for each type of link (card, address, email, device, and so on).
- **HAN:** attention over neighbours and also over link types, so the model learns which entity type matters most.
- **TabNet:** the reference paper's tabular model; at each step it chooses which columns to look at (attention masks).

Day 1: what gets sent

Only three things are shared: this plan, the split file and the SOA graph. Everything else each member builds alone.

From	To	What
Reset	Everyone	This plan (PDF) and `ieee_sorted_split.parquet`

From	To	What
SOA member	FT-Transformer teammate, new members 1 and 2	The graph files below, built from Reset's split file
Every member	Reset, through the shared results folder	Score and history files as runs finish (see "What every member hands in")

The graph files (from the SOA member):

File	Contents
`graph_edges.npy`	The links between entity nodes, each link once
`node_type.npy`	The entity type of every node
`payment_nodes_train.parquet`, `payment_nodes_val.parquet`, `payment_nodes_test.parquet`	TransactionID, isFraud and the 6 entity node numbers of every payment
`graph_shuf_seed42_edges.npy`, `graph_shuf_seed1_edges.npy`, `graph_shuf_seed2_edges.npy`	The three shuffled link files
`graph_info.json`	Entity columns, node and link counts, and the "missing" and "unknown" node numbers for each entity type

The files are saved as .npy, .parquet and .json rather than pickle, so they open safely on any computer.

Tasks by member

Every training run uses the fixed setup above (and every graph model the mandatory graph settings) and saves its scores as described in "What every member hands in". Tick each task when it is done.

Reset

- [ ] Day 1: send everyone `ieee_sorted_split.parquet` and this plan, and set up the shared results folder.
- [x] ~~THGNN with and without V, 3 seeds each (done).~~

- [ ] Plain network (THGNN without graph and history) on column sets A and B, 3 seeds each, saving validation and test scores (6 runs, about 2 min each).
- [ ] LightGBM on column sets A and B, as the strongest non-neural reference (2 runs, about 5 min each).
- [ ] Graph-features baseline: for each payment, how many training payments share its card, address, email and device, and their fraud rate (training payments only). Add these as extra columns to the plain network, on A and B, 3 seeds each (6 runs).
- [ ] Export THGNN's test scores with TransactionID, in the same file format as everyone else.
- [ ] Fusion and evaluation (no training): combine each tabular model (plain network, FT-Transformer, TabNet) with each graph model (GraphSAGE, GAT, R-GCN, HAN), run the shuffled-graph comparisons, compare with the paper's hybrid, and make the results tables and figures. Start now on the files that already exist; re-run when new files arrive.
- [ ] Put the paper together.

FT-Transformer teammate

- [ ] Switch early stopping to validation ROC-AUC (the paper's metric) with patience 10, and check the preprocessing follows the paper's Section 3.2.
- [ ] FT-Transformer on column sets A and B, 3 seeds each, saving validation and test scores (6 runs).
- [ ] GraphSAGE on the SOA member's graph file: build and test it his own way, free to try other settings. Final runs: 3 seeds, saving validation and test scores (3 runs), and 3 seeds on the shuffled link files (3 runs).
- [ ] Write the method paragraph for FT-Transformer and GraphSAGE, including the GraphSAGE settings he chose.

New member 1: GAT

- [ ] Load the shared split file and the SOA member's graph files; check the graph has 14,239 nodes, as in `graph_info.json`.
- [ ] Build GAT following "Mandatory settings for all graph models"; only the layer differs (attention over neighbours).
- [ ] Train GAT, 3 seeds, saving validation and test scores (3 runs).
- [ ] Train GAT on the shuffled link files, 3 seeds (3 runs).
- [ ] Write the method paragraph for GAT, including the number of attention heads and the other free choices.

New member 2: R-GCN

- [ ] Load the shared split file and the SOA member's graph files; check the graph has 14,239 nodes, as in `graph_info.json`.
- [ ] Build R-GCN following "Mandatory settings for all graph models"; only the layer differs (one set of weights per link type).
- [ ] Train R-GCN, 3 seeds, saving validation and test scores (3 runs).
- [ ] Train R-GCN on the shuffled link files, 3 seeds (3 runs).
- [ ] Write the method paragraph for R-GCN, including the free choices.

SOA member: graph file, HAN, TabNet and the paper's hybrid

- [ ] Day 1, graph: build the entity graph with his existing code (`build_entity_index`, `map_entities`, `build_edges`) from Reset's `ieee_sorted_split.parquet` instead of his own split, and check it gives 14,239 nodes and 12,401,340 links before repeats are removed. If the counts differ, stop and compare the split with Reset before anyone trains.
- [ ] Day 1, graph: make the three shuffled link files (seeds 42, 1 and 2): for each entity column, shuffle the training payments' values at random, then rebuild the links. The payment node files stay real.
- [ ] Day 1, graph: save the graph files (see "Day 1: what gets sent") as .npy, .parquet and .json, with TransactionID in the payment files, and send them to the FT-Transformer teammate and new members 1 and 2.
- [ ] HAN: build it following "Mandatory settings for all graph models"; only the layer differs (attention over neighbours and over entity types).
- [ ] HAN: train 3 seeds, saving validation and test scores (3 runs), then 3 seeds on the shuffled link files (3 runs).
- [ ] TabNet: load Reset's split file instead of building his own split, and add `split` to the dropped columns.
- [ ] TabNet: keep only column set A or B (re-run his preprocessing for each set, so the categorical sizes are rebuilt).
- [ ] TabNet: keep his TabNet settings (n_d = n_a = 32, 5 steps, lr 0.02, batch 8192, patience 10, early stopping on validation ROC-AUC) and run seeds 42, 1 and 2 on A and B (6 runs, about 7 min each).
- [ ] TabNet: save validation and test scores as logits with TransactionID, plus the history file.
- [ ] Paper's hybrid: TabNet + GraphSAGE joined into one TabNet classifier, as in the paper's Fig. 4, using his notebook 04b with the leak fix (DECISIONS.md, D18), the shared split file and graph, column sets A and B, 3 seeds (6 runs). Save scores like everyone else.

- [ ] Write the method paragraphs for HAN, TabNet and the paper's hybrid, including which entity types HAN weights most.

What gets compared

Each comparison changes one thing and answers one research question.

Comparison	What changes	Answers
Tabular model alone vs tabular model + graph model (3 tabular models × 4 graph models, column sets A and B)	Adding the graph	RQ1
Gain from the graph with column set A vs with column set B	Removing the V columns	RQ2
Tabular + real graph vs tabular + shuffled graph	Whether the links are real	RQ3
THGNN vs the same network without graph and history, A and B	THGNN's graph and history, learned inside one model	RQ1, RQ2
The paper's TabNet–GraphSAGE hybrid vs TabNet alone, A and B	The paper's own way of combining the two models	RQ1, RQ2, and a direct check of the paper
Plain network + graph features vs plain network alone	The graph's information given as simple columns	Whether a graph model is needed at all
LightGBM on A and B	Nothing (reference)	How strong a non-neural model is

Combining a tabular model with a graph model (late fusion). Both models are trained on their own. Their scores are then combined:

score = z(tabular) + α × z(graph)

Each z is that model's score, standardised with the mean and std of its own validation scores. α goes from 0 to 1 in steps of 0.025 and is picked on validation PR-AUC only; α = 0 means the tabular model alone. Seeds are paired: seed 42 of the tabular model with seed 42 of the graph model, and so on. The paper itself joins the two models' learned features inside one TabNet classifier; that design is tested separately through the SOA member's hybrid.

The shuffled graph. For each entity column, the training payments' values are shuffled at random (seed 42, 1 or 2), and the links are rebuilt from these shuffled payments. Every entity keeps how many payments use it, but which entities are linked becomes random. Each payment still uses its own real entities. The SOA member makes the three shuffled link files, so all four graph models use exactly the same ones.

How many runs. 56 training runs in total, plus 144 combinations that need no training (3 tabular models × 2 column sets × 4 graph models × real or shuffled × 3 seeds).

Member	Training runs
Reset	14 (plain network 6, LightGBM 2, graph features 6)
FT-Transformer teammate	12 (FT-Transformer 6, GraphSAGE 3, shuffled 3)
New member 1 (GAT)	6 (GAT 3, shuffled 3)
New member 2 (R-GCN)	6 (R-GCN 3, shuffled 3)
SOA member (HAN, TabNet, hybrid)	18 (HAN 3, shuffled 3, TabNet 6, hybrid 6)

How we measure

A graph counts as helping only if it passes all three checks below; anything less is reported as no gain.

What is measured, on the test set:

- ROC-AUC and PR-AUC (average precision), the reference paper's two metrics.
- 95% confidence intervals from a paired bootstrap of the test payments (2,000 resamples).
- The mean and range over the 3 seeds, never only the best seed.
- Review budget: the share of frauds caught if a team reviews the top 1% and the top 5% of test payments by score.

A gain is real only if all three hold:

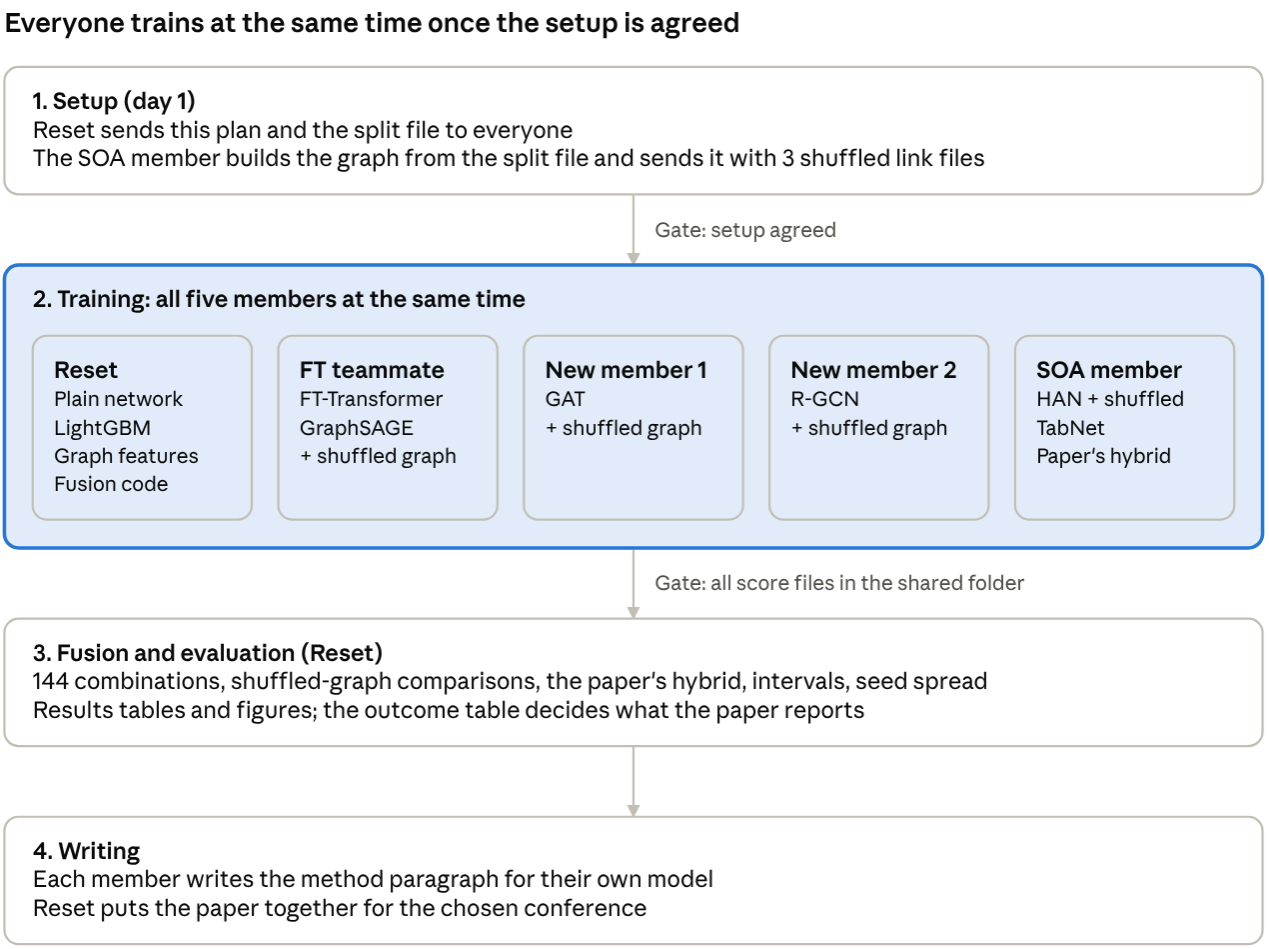
1. The 95% interval of the difference (tabular + graph minus tabular alone) lies above 0.
2. The gain is larger than the spread between the 3 seeds.
3. The real graph beats the shuffled graph, by more than chance under the same bootstrap.

What the results will mean

Every possible outcome gives the paper a clear finding; THGNN already shows the first row.

What we see	What it means	What the paper says
No graph model helps, with or without V, and the real graph ≈ the shuffled graph	The links carry no fraud information beyond each payment's own columns, on this data and setup	Graph models add nothing on IEEE-CIS under the reference paper's setup, shown with five graph designs instead of one
Graph models help only without V	The V columns already hold what the graph knows	The V columns act as a hidden graph; the graph only matters when they are missing
Graph models help with and without V, and beat the shuffled graph	The graph adds real information; our first two results were specific to those models	Which graph models help, by how much, and why THGNN and GraphSAGE did not
Only some graph models help	The type of layer matters	What those models do differently (attention, link-type weights)
The paper's hybrid helps but score fusion does not	The gain depends on how the two models are combined	Which way of combining the gain needs, compared with the paper's design
Graph features help but graph models do not	The useful part is simple counts, not message passing	A cheaper model is enough

Order of work



order of work · 4 phases, 2 gates

Nobody waits for another member's code: the only shared inputs are the split file and the graph file, both sent on day 1. Reset builds the fusion and evaluation during phase 2, using the scores that already exist.

What every member hands in

Each training run puts three files into the shared results folder, named so anyone can tell what the run was. Keeping TransactionID in every score file is what lets us combine models made by different people, row by row.

File	Contents
`<run>_val.parquet`	TransactionID, isFraud, score for all 88,581 validation payments
`<run>_test.parquet`	TransactionID, isFraud, score for all 88,581 test payments

File	Contents
`<run>_history.json`	Per epoch: loss, validation ROC-AUC, validation PR-AUC. Plus the run's model, column set, seed, settings and chosen epoch

Also, one row per run in the shared results table: run name, test ROC-AUC, test PR-AUC, best epoch, minutes.

Run names:

- Tabular models: `<model>_<column set>_seed<N>`, for example `ft_A_seed42`, `plain_B_seed1` or `tabnet_A_seed2`.
- Graph models have no column set: `<model>_seed<N>`, for example `gat_seed2`, `rgcn_seed1` or `han_seed42`.
- Shuffled graph: `<model>_shuf_seed<N>`, for example `han_shuf_seed42`.
- The paper's hybrid: `hybrid_<column set>_seed<N>`, for example `hybrid_B_seed1`.

Decisions the team still needs to make

Already settled: model selection on validation ROC-AUC (the paper's Table 3), the SOA member's graph, the shuffle rule, and 2 graph layers. Two decisions are still open.

- [ ] **Conference and deadline:** sets how many runs fit and which paper template to use.
- [ ] **Column set C:** also test all 431 columns, or only A and B.